Boolean Satisfiability

Edward A. Hirsch

http://edwardahirsch.github.io/edwardahirsch

Ariel University

January 8, 2024

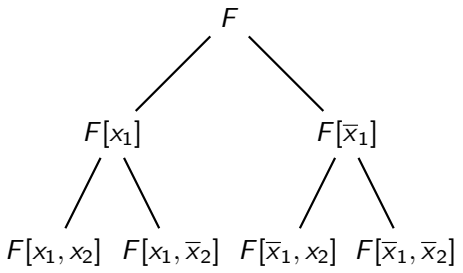
Fighting Perebor (brute-force search): Lecture overview

1. Divide-and-conquer for ***k*-SAT**.
2. Generalization for **SAT**.
3. Two randomized methods for ***k*-SAT**: random permutations and local search.
4. Getting rid of the randomness.
5. Quantum?

Fighting Perebor (brute-force search)

Weakly exponential **SAT** algorithms; start with **3-SAT**

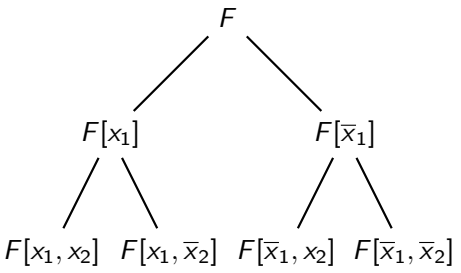
Can we do better than 2^n?



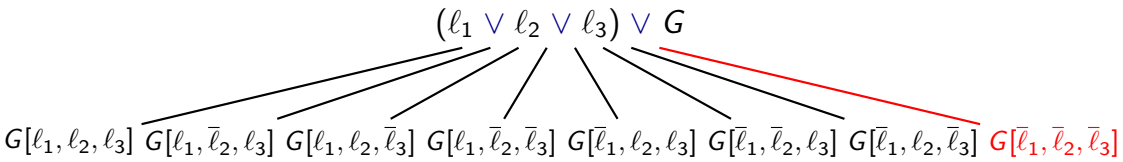
Fighting Perebor (brute-force search)

Weakly exponential **SAT** algorithms; start with **3-SAT**

Can we do better than 2^n?



The first attempt,
splitting over a 3-clause.



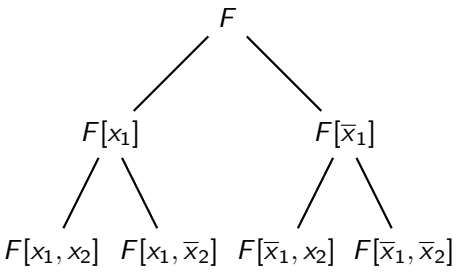
$T(F) :=$ **#leaves**, $\quad T(n) := \max_{\mathrm{vars}(F)=n} T(F)$.

$T(n) \leqslant 7 \cdot T(n-3) \leqslant \ldots \leqslant 7^{\lceil n/3 \rceil} = O(1.92^n)$.

Fighting Perebor (brute-force search)

Weakly exponential **SAT** algorithms; start with **3-SAT**

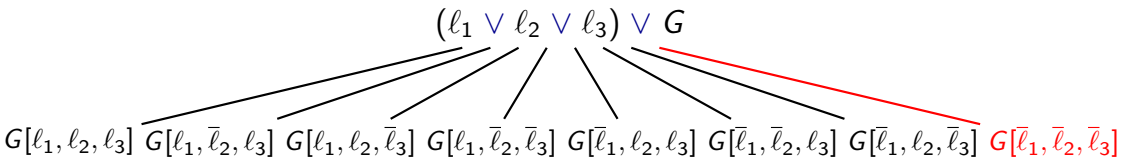
Can we do better than 2^n?



The first attempt,
splitting over a 3-clause.

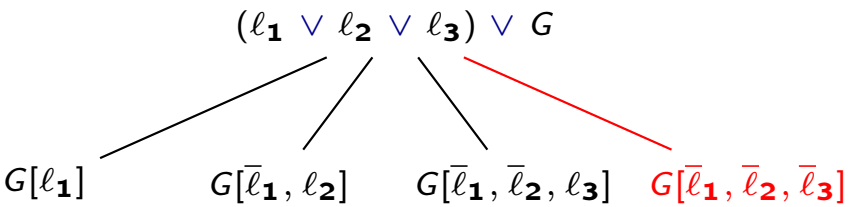
$T(F) :=$ #leaves, $\quad T(n) := \max_{\mathrm{vars}(F)=n} T(F)$.

$T(n) \leqslant 7 \cdot T(n-3) \leqslant \ldots \leqslant 7^{\lceil n/3 \rceil} = O(1.92^n)$.



The second attempt.

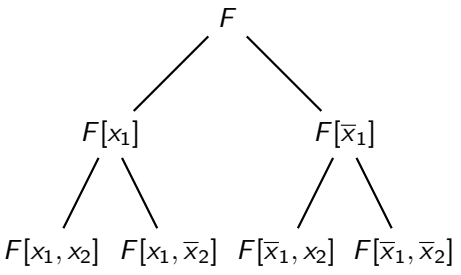
$T(n) \leqslant T(n-1) + T(n-2) + T(n-3) \leqslant ???$



Fighting Perebor (brute-force search)

Weakly exponential **SAT** algorithms; start with **3-SAT**

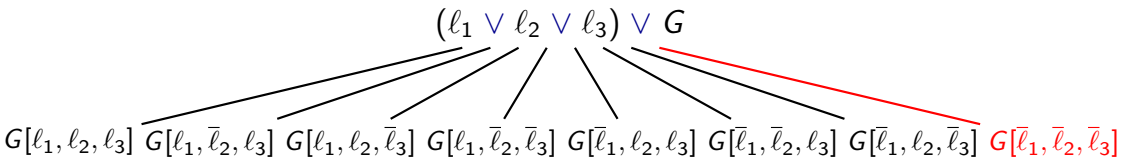
Can we do better than 2^n?



The first attempt,
splitting over a 3-clause.

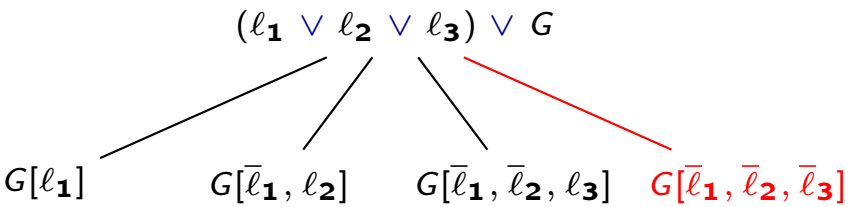
$T(F) :=$#leaves, $\quad T(n) := \max_{\mathrm{vars}(F)=n} T(F)$.

$T(n) \leqslant 7 \cdot T(n-3) \leqslant \ldots \leqslant 7^{\lceil n/3 \rceil} = O(1.92^n)$.



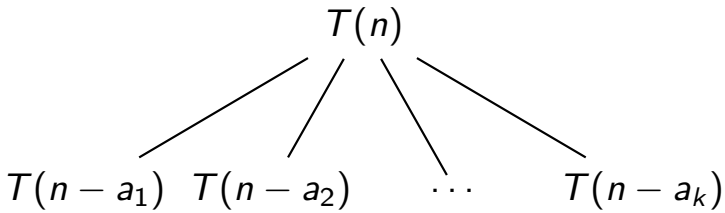
The second attempt.

$T(n) \leqslant T(n-1) + T(n-2) + T(n-3) \leqslant O(1.84^n)$,
but why?



Estimating the recurrency

Assume $k \geqslant 2$, $T(n) \leqslant \sum_{i=1}^{k} T(n-a_i)$.
Call $\vec{a} = (a_1, \ldots, a_k) \in \mathbb{N}_{\geqslant 1}^k$ a branching tuple.
Call $\chi_{\vec{a}}(x) = 1 - \sum_{i=1}^{k} x^{-a_i}$ its characteristic polynomial.
Let $\tau_{\vec{a}}$ be its (only) positive root, branching number.



Lemma (branching numbers)

Let $\mu\colon \{formulas\} \to \mathbb{R}_{\geqslant 0}$, $\mu(leaves) = 0$.
Let $\tau = \max_{\vec{a}} \tau(\vec{a})$ for all $\vec{a}$ in the tree (wrt μ).
Then $T(F) \leqslant \tau^{\mu(F)}$.

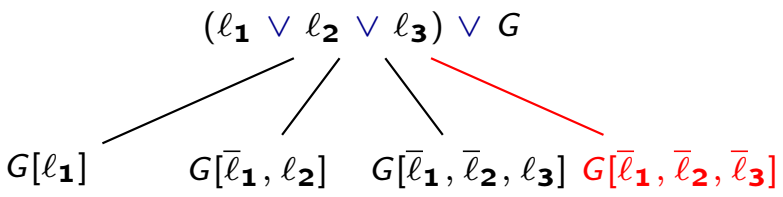
Proof (induction).

$$T(F) \leqslant \sum_{j=1}^{k} T(F_j) \overset{(ind)}{\leqslant} \sum_{j=1}^{k} \tau^{\mu(F)-a_j} = \tau^{\mu(F)} \cdot \sum_{j=1}^{k} \tau^{-a_j} = \tau^{\mu(F)}(1 - \chi_{\vec{t}}(\tau)) = \tau^{\mu(F)}.$$

3-SAT algorithm, revisited

The second attempt.

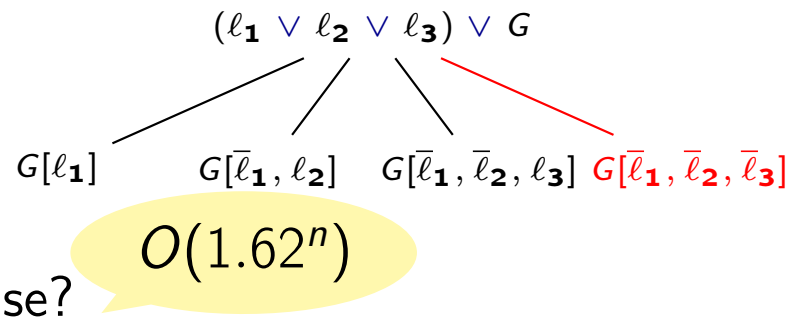
$T(n) \leqslant T(n-1) + T(n-2) + T(n-3)$, branching number $\leqslant 1.84$
$T(n) \leqslant T(n-1) + T(n-2)$, branching number $\phi \leqslant 1.62$
Thus $O(1.84^n)$ leaves.



3-SAT algorithm, revisited

The second attempt.

$T(n) \leqslant T(n-1) + T(n-2) + T(n-3)$, branching number $\leqslant 1.84$
$T(n) \leqslant T(n-1) + T(n-2)$, branching number $\phi \leqslant 1.62$
Thus $O(1.84^n)$ leaves.



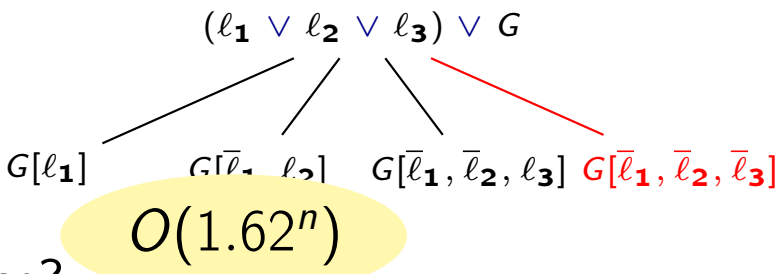
$O(1.62^n)$

The third attempt: Can we always find a 2-clause?

3-SAT algorithm, revisited

The second attempt.

$T(n) \leqslant T(n-1) + T(n-2) + T(n-3)$, branching number $\leqslant 1.84$
$T(n) \leqslant T(n-1) + T(n-2)$, branching number $\phi \leqslant 1.62$
Thus $O(1.84^n)$ leaves.



The third attempt: Can we always find a 2-clause?

Davis, Putnam, Logemann, Loveland

General DPLL algorithm:

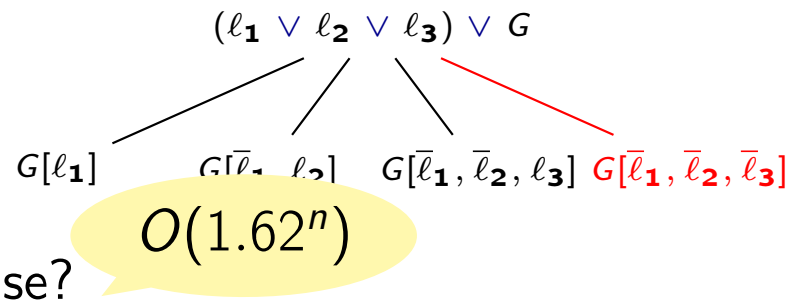
Split. $F \mapsto F_1, F_2, \ldots, F_k$.

Reduce. Simplify F_i's (e.g., by unit propagation).

3-SAT algorithm, revisited

The second attempt.

$T(n) \leqslant T(n-1) + T(n-2) + T(n-3)$, branching number $\leqslant 1.84$
$T(n) \leqslant T(n-1) + T(n-2)$, branching number $\phi \leqslant 1.62$
Thus $O(1.84^n)$ leaves.



The third attempt: Can we always find a 2-clause?

Davis, Putnam, Logemann, Loveland

General DPLL algorithm:

Split. $F \mapsto F_1, F_2, \ldots, F_k$.

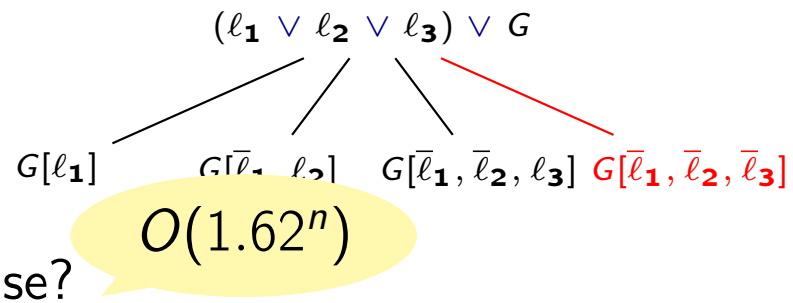
Reduce. Simplify F_i's (e.g., by unit propagation).

- Assume **Reduce**$(F_i) = F[A]$ and suppose it contains no 2-clauses.
- Then $F[A] \subseteq F$.
- Very good!
- Replace F by $F[A]$ instead of splitting it.

3-SAT algorithm, revisited

The second attempt.

$T(n) \leqslant T(n-1) + T(n-2) + T(n-3)$, branching number $\leqslant 1.84$
$T(n) \leqslant T(n-1) + T(n-2)$, branching number $\phi \leqslant 1.62$
Thus $O(1.84^n)$ leaves.



The third attempt: Can we always find a 2-clause?

Davis, Putnam, Logemann, Loveland

General DPLL algorithm:

Split. $F \mapsto F_1, F_2, \ldots, F_k$.

Reduce. Simplify F_i's (e.g., by unit propagation).

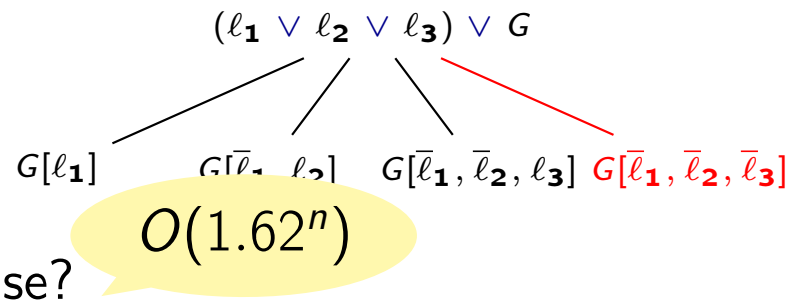
- Assume **Reduce**$(F_i) = F[A]$ and suppose it contains no 2-clauses.
- Then $F[A] \subseteq F$.
- Very good!
- Replace F by $F[A]$ instead of splitting it.

Why am I writing $O(\ldots)$ here?

3-SAT algorithm, revisited

The second attempt.

$T(n) \leqslant T(n-1) + T(n-2) + T(n-3)$, branching number $\leqslant 1.84$
$T(n) \leqslant T(n-1) + T(n-2)$, branching number $\phi \leqslant 1.62$
Thus $O(1.84^n)$ leaves.



The third attempt: Can we always find a 2-clause?

Davis, Putnam, Logemann, Loveland

General DPLL algorithm:

Split. $F \mapsto F_1, F_2, \ldots, F_k$.

Reduce. Simplify F_i's (e.g., by unit propagation).

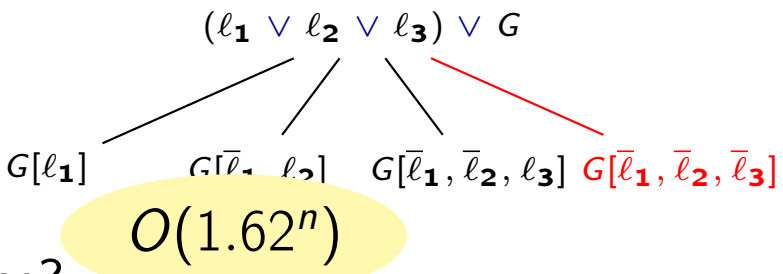
- Assume **Reduce**$(F_i) = F[A]$ and suppose it contains no 2-clauses.
- Then $F[A] \subseteq F$.
- Very good!
- Replace F by $F[A]$ instead of splitting it.

Why am I writing $O(\ldots)$ here? First step!

3-SAT algorithm, revisited

The second attempt.

$T(n) \leqslant T(n-1) + T(n-2) + T(n-3)$, branching number $\leqslant 1.84$
$T(n) \leqslant T(n-1) + T(n-2)$, branching number $\phi \leqslant 1.62$
Thus $O(1.84^n)$ leaves.



The third attempt: Can we always find a 2-clause?

Davis, Putnam, Logemann, Loveland

General DPLL algorithm:

Split. $F \mapsto F_1, F_2, \ldots, F_k$.

Reduce. Simplify F_i's (e.g., by unit propagation).

- Assume **Reduce**$(F_i) = F[A]$ and suppose it contains no 2-clauses.
- Then $F[A] \subseteq F$.
- Very good!
- Replace F by $F[A]$ instead of splitting it.

Why am I writing $O(\ldots)$ here? First step!
Time $\widetilde{O}(\phi^n)$, where $f(n) = \widetilde{O}(g(n))$ if $f(n) = O(g(n) \cdot \mathrm{size}(F)^k)$ for some $k \in \mathbb{N}$.

Other tricks to extend DPLL

Pure literal. If $\ell \in F$, $\overline{\ell} \notin F$, then $F := F[\ell]$.

Subsumption. If $C \subseteq D$, then remove D.

Equivalent literals. If F contains $\overline{\ell} \vee \ell'$ and $\overline{\ell}' \vee \ell$, then eliminate ℓ' and $\overline{\ell}'$ by replacing them with ℓ and $\overline{\ell}$, resp.

Blocked clause. A literal ℓ in a clause C blocks it if
$\forall D \in F\ (\overline{\ell} \in D \implies |C \cap \overline{D}| \geqslant 2)$.
Then we can remove C. (Prove! See HW1.1(a).)

Amortized analysis. Choose $\mu(F) = n - \alpha \cdot \#\{\text{variables in 2-clauses}\}$, etc.

Other tricks to extend DPLL

Pure literal. If $\ell \in F$, $\overline{\ell} \notin F$, then $F := F[\ell]$.

Subsumption. If $C \subseteq D$, then remove D.

Equivalent literals. If F contains $\overline{\ell} \vee \ell'$ and $\overline{\ell}' \vee \ell$, then eliminate ℓ' and $\overline{\ell}'$ by replacing them with ℓ and $\overline{\ell}$, resp.

Blocked clause. A literal ℓ in a clause C blocks it if
$\forall D \in F\ (\overline{\ell} \in D \implies |C \cap \overline{D}| \geqslant 2)$.
Then we can remove C. (Prove! See HW1.1(a).)

Amortized analysis. Choose $\mu(F) = n - \alpha \cdot \#\{\text{variables in 2-clauses}\}$, etc.

Definition (Resolvent)

Given $x \vee C$ and $\overline{x} \vee D$ s.t. $C \cap \overline{D} = \emptyset$, their resolvent (by x) is $C \vee D$.

- Semantically implied by them.
- Davis–Putnam procedure (single step), $\mathrm{DP}_x(F)$:
  - Add all resolvents by x to F.
  - Remove all clauses containing x or $\overline{x}$.

  The resulting formula is satisfiability-equivalent to F.

  The satisfying assignment can be reconstructed.

  (Prove both, see HW1.1(b).)

Homework Assignment #1, start...

Problem (HW1.1)

Prove that the following transformations are satisfiability-equivalent[1]

(a) *Blocked clause removal.*

(b) $DP_x(F)$.

[1] $F \overset{\mathbf{SAT}}{\equiv} G$ if $F \in \mathbf{SAT} \Leftrightarrow G \in \mathbf{SAT}$.
$T(\cdot)$ is satisfiability-equivalent if $\forall F\ \mathbf{SAT}(F) \overset{\mathbf{SAT}}{\equiv} T(F)$.

Homework Assignment #1, start...

Problem (HW1.2)

Consider the satisfiability problem for formulas that can be expressed as CSPs for relations of two types:

- *clauses containing at most two variables,*
- *xor-equations containing at most six variables.*

Using the DPLL approach, design an algorithm for this problem with an exponent strictly better than ϕ^n (such as 1.6^n). Prove its correctness and the upper bound on its running time.

Homework Assignment #1, start...

Problem (HW1.3)

Assume that we are given a formula in 3-CNF such that at least 1% *of the complete assignments satisfy it. Find a satisfying assignment in a polynomial time without the use of randomness. (Of course, provide a proof.)*

There will be more problems in the assignment!
Even more than you need to get 100% for it.

Making DPLL practical

- Upper bounds generalize to ***k*-SAT**.
- The same tricks work for **SAT**.
- DPLL is not currently the best in theory, but it can be made very practical:
  - clause learning,
  - optimized data structures.

Making DPLL practical

- Upper bounds generalize to ***k*-SAT**.
- The same tricks work for **SAT**.
- DPLL is not currently the best in theory,
  but it can be made very practical:
  - clause learning,
  - optimized data structures.

Clause learning is a technique for adding useful resolvents.
CDCL solvers: later in the course.

Optimized data structures are "lazy":
certain updates are delayed until the data is used (also later in the course).

Classically, one represents a CNF using adjacency lists:

- A clause is a list of literals.
- For every variable, there is a list of clauses where it occurs.

Pro: Enough for theoretical results and for the use in "proof-of-concept" solvers.
Contra: A lot of data to update after assigning a single variable.

Randomizing the formula

Observation

Consider a uniquely satisfiable formula (1 sat. assign.) satisfied by A.

- *For each variable* $\boldsymbol{x}$*, there* ***must*** *be a clause of* `--+` *pattern* ($C = \ell_1 \vee \ell_2 \vee \ell_3$*, for* $\ell_3 \in \{x, \overline{x}\}$ *and* $A[\ell_1] = A[\ell_2] = \texttt{False},\ A[\ell_3] = \texttt{True}$*).*
- *The variable* $\boldsymbol{x}$ *is critical for* C*.*
- *Then if we know the values of* ℓ_1, ℓ_2*, we know* x*, if it comes later.*

Randomizing the formula

Observation

Consider a uniquely satisfiable formula (1 sat. assign.) satisfied by A.

- *For each variable* $\boldsymbol{x}$*, there* ***must*** *be a clause of* $--+$ *pattern* ($C = \ell_1 \vee \ell_2 \vee \ell_3$, *for* $\ell_3 \in \{x, \overline{x}\}$ *and* $A[\ell_1] = A[\ell_2] = \texttt{False}$, $A[\ell_3] = \texttt{True}$).
- *The variable* $\boldsymbol{x}$ *is critical for* C.
- *Then if we know the values of* ℓ_1, ℓ_2*, we know* x*, if it comes later.*

- If we take the order at random, the critical variable in C is the last with prob. $1/|C|$.
- $\boldsymbol{E}(\#\text{values given for free}) \geqslant n/k$ for $\boldsymbol{k}$**-SAT**.

PPZ: the uniquely satisfiable case

Algorithm (Paturi, Pudlák, Zane)

```
Initialize A[1..n] by *.
Pick a random permutation π ∈ S_n.
For i = 1,...,n
   If A[x_π(i)] = * then
         A[x_π(i)] := random({0,1})                                  (G)
         F := F[x_π(i) ← A[x_π(i)]]
         Perform unit propagation (updating A))
If ∅ ∉ A then return A else return "no"
```

- The probability of a correct guess in (G) is $1/2$.
- $\boldsymbol{E}(\#\text{values given for free}) \geqslant n/k$ for $\boldsymbol{k}$-**SAT**.
- This $\# \geqslant n/k$ with prob. $\geqslant 1/n$, otherwise

$$\boldsymbol{E} < n \cdot 1/n + (n/k - 1) \cdot (1 - 1/n) = 1 + (n/k - 1/k) - (1 - 1/n) < n/k$$

(assume $n \equiv 0 \pmod{k}$).

PPZ: the uniquely satisfiable case

Algorithm (Paturi, Pudlák, Zane)

```
Initialize A[1..n] by *.
Pick a random permutation π ∈ S_n.
For i = 1,...,n
   If A[x_π(i)] = * then
         A[x_π(i)] := random({0,1})                                  (G)
         F := F[x_π(i) ← A[x_π(i)]]
         Perform unit propagation (updating A))
If ∅ ∉ A then return A else return "no"
```

- The probability of a correct guess in (G) is $1/2$.
- $\boldsymbol{E}(\#\text{guesses}) \leqslant n - n/k$.
- $(\#\text{guesses}) \leqslant n - n/k$ with prob. $\geqslant 1/n$.

PPZ: the uniquely satisfiable case

Algorithm (Paturi, Pudlák, Zane)

```
Initialize A[1..n] by *.
Pick a random permutation π ∈ S_n.
For i = 1,...,n
   If A[x_π(i)] = * then
         A[x_π(i)] := random({0,1})                                  (G)
         F := F[x_π(i) ← A[x_π(i)]]
         Perform unit propagation (updating A))
If ∅ ∉ A then return A else return "no"
```

- The probability of a correct guess in (G) is $1/2$.
- $\boldsymbol{E}(\#\text{guesses}) \leqslant n - n/k$.
- $(\#\text{guesses}) \leqslant n - n/k$ with prob. $\geqslant 1/n$.
- The overall success probability is $\geqslant 2^{-(n-n/k)} \cdot 1/n$.
- Repeat it $O(n \cdot 2^{n-n/k})$ times to get a constant probability of error.

Reminder: if a one-sided error algorithm has success probability $\geqslant p$, repeating it $1/p$ times fails with prob. $\leqslant (1-p)^{1/p} < 1/\boldsymbol{e}$.

PPZ: Satisfiability Coding Lemma

- Intuitively, the more sat. assignments, the easier the problem.

PPZ: Satisfiability Coding Lemma

- Intuitively, the more sat. assignments, the easier the problem.
- Let A^x be A with x's value flipped ($0 \leftrightarrow 1$).
- An assignment A is d-isolated if there are d variables x s.t. $F[A^x] = 0$.

PPZ: Satisfiability Coding Lemma

- Intuitively, the more sat. assignments, the easier the problem.
- Let A^x be A with x's value flipped ($0 \leftrightarrow 1$).
- An assignment A is d-isolated if there are d variables x s.t. $F[A^x] = 0$.

$F = (\overline{x}_1 \vee \overline{x}_2 \vee x_3) \wedge (x_1 \vee x_4) \wedge (x_2 \vee x_4) \wedge (\overline{x}_3 \vee \overline{x}_4)$

A, in full: `[1, 1, 1, 0]`

A, compressed : `[1, 1, *, *]`

PPZ: Satisfiability Coding Lemma

- Intuitively, the more sat. assignments, the easier the problem.
- Let A^x be A with x's value flipped ($0 \leftrightarrow 1$).
- An assignment A is d-isolated if there are d variables x s.t. $F[A^x] = 0$.

$F = (\overline{x}_1 \vee \overline{x}_2 \vee x_3) \wedge \cancel{(x_1 \vee x_4)} \wedge (x_2 \vee x_4) \wedge (\overline{x}_3 \vee \overline{x}_4)$

A, in full: `[1, 1, 1, 0]`

A, compressed : `[1, 1, *, *]`

PPZ: Satisfiability Coding Lemma

- Intuitively, the more sat. assignments, the easier the problem.
- Let A^x be A with x's value flipped ($0 \leftrightarrow 1$).
- An assignment A is d-isolated if there are d variables x s.t. $F[A^x] = 0$.

$F = (\overline{x}_1 \vee \overline{x}_2 \vee x_3) \wedge \cancel{(x_1 \vee x_4)} \wedge \cancel{(x_2 \vee x_4)} \wedge (\overline{x}_3 \vee \overline{x}_4)$

A, in full: `[1, 1, 1, 0]`

A, compressed : `[1, 1, *, *]`

PPZ: Satisfiability Coding Lemma

- Intuitively, the more sat. assignments, the easier the problem.
- Let A^x be A with x's value flipped ($0 \leftrightarrow 1$).
- An assignment A is d-isolated if there are d variables x s.t. $F[A^x] = 0$.

$F = \cancel{(\overline{x}_1 \vee \overline{x}_2 \vee \boxed{x_3})} \wedge \cancel{(x_1 \vee x_4)} \wedge \cancel{(x_2 \vee x_4)} \wedge (\overline{x}_3 \vee \overline{x}_4)$

A, in full: `[1, 1, 1, 0]`

A, compressed : `[1, 1, *, *]`

PPZ: Satisfiability Coding Lemma

- Intuitively, the more sat. assignments, the easier the problem.
- Let A^x be A with x's value flipped ($0 \leftrightarrow 1$).
- An assignment A is d-isolated if there are d variables x s.t. $F[A^x] = 0$.

$F = \cancel{(\overline{x}_1 \vee \overline{x}_2 \vee \boxed{x_3})} \wedge \cancel{(x_1 \vee x_4)} \wedge \cancel{(x_2 \vee x_4)} \wedge \cancel{(\overline{x}_3 \vee \boxed{\overline{x}_4})}$

A, in full: `[1, 1, 1, 0]`

A, compressed : `[1, 1, *, *]`

1 0

PPZ: Satisfiability Coding Lemma

- Intuitively, the more sat. assignments, the easier the problem.
- Let A^x be A with x's value flipped ($0 \leftrightarrow 1$).
- An assignment A is d-isolated if there are d variables x s.t. $F[A^x] = 0$.

$F = (\bar{x}_1 \vee \bar{x}_2 \vee \boxed{x_3}) \wedge (x_1 \vee x_4) \wedge (x_2 \vee x_4) \wedge (\bar{x}_3 \vee \boxed{\bar{x}_4})$

A, in full: `[1, 1, 1, 0]`

A, compressed : `[1, 1, *, *]`

1 0

Lemma (Satisfiability Coding Lemma for k-SAT)

Sat. assignment A is d-isolated $\Rightarrow$ $\boldsymbol{E}\#$guesses needed to recover it $\leqslant n - d/k$.

- If A is isolated wrt x, then x has a critical clause C.
- There are d such variables, their clauses are different.
- x is the last in C with prob. $\geqslant 1/k$,
- $\boldsymbol{E}\#(\text{such variables}) \geqslant d \cdot 1/k$.
- $\boldsymbol{E}\#(\text{guesses}) \leqslant n - d \cdot 1/k$, at most $\boldsymbol{E}$ with prob. $\geqslant 1/n$.

PPZ: The number of (non-)isolated solutions.

If A is d-isolated in F, let $\delta(A) := \delta_F(A) := d$.
(Recall that $\#(\text{guesses}) \leqslant n - \delta(A)/k$ with prob. $\geqslant 1/n$.)

Lemma (Isolated sat. assignments of formula F)

Then $\displaystyle\sum_{A\colon F[A]=1} 2^{\delta(A)-n} \geqslant 1.$

Induction on dimension: If true for $F[x \leftarrow \texttt{False}]$ and $F[x \leftarrow \texttt{True}]$, then

- If both formulas are **SAT**, then
$$\sum_{A\colon F[A]=1} 2^{\delta(A)-n} = \sum_{A\ni x\colon F[A]=1} 2^{\delta(A)-n} + \sum_{A\ni \overline{x}\colon F[A]=1} 2^{\delta(A)-n} \geqslant$$
$$\sum_{B\colon F[x\leftarrow 1][B]=1} 2^{\delta_\ldots(B)-n+1} + \sum_{C\colon F[x\leftarrow 0][C]=1} 2^{\delta_\ldots(C)-n+1} \overset{\textit{ind}}{\geqslant} \tfrac{1}{2} + \tfrac{1}{2} = 1.$$
- If one is **UNSAT**, then for the other one (G), $\delta_G(A \setminus \{x, \overline{x}\}) = \delta_F(A) - 1$,
so $\displaystyle\sum_{A\colon F[A]=1} 2^{\delta(A)-n} = \sum_{B\colon G[B]=1} 2^{\delta_G(B)-1-n+1} \overset{\textit{ind}}{\geqslant} 1.$

PPZ: the general case

Algorithm (Paturi, Pudlák, Zane)

```
Initialize A[1..n] by *.
Pick a random permutation π ∈ S_n.
For i = 1, ..., n
   If A[x_π(i)] = * then
        A[x_π(i)] := random({0,1})                                  (G)
        F := F[x_π(i) ← A[x_π(i)]]
        Perform unit propagation (updating A)
```

Let $S := \{\text{set of all sat. assign. for } F\}$.

$$\Pr_{A\in S}\{\text{success}\} \overset{\text{non-intersecting}}{=} \sum_{A\in S} \Pr\{A \text{ is output}\} \geqslant \sum_{A\in S} \frac{1}{n}\cdot 2^{\delta(A)/k-n} =$$

$$= \frac{1}{n}\cdot 2^{n/k-n}\cdot \sum_{A\in S} 2^{(\delta(A)-n)/k} \geqslant \qquad (\sqrt[k]{\varepsilon} \geqslant \varepsilon \text{ for } 1 \geqslant \varepsilon \geqslant 0)$$

$$\geqslant \frac{1}{n}\cdot 2^{n/k-n}\cdot \sum_{A\in S} 2^{\delta(A)-n} \geqslant \qquad \text{(last Lemma)}$$

$$\geqslant \frac{1}{n}\cdot 2^{n/k-n}.$$

Time $\widetilde{O}(2^{n-n/k})$ for ***k*-SAT**, $\widetilde{O}(2^{2n/3})$ for **3-SAT**.

Enhancement: PPSZ (+Michael Saks)

Algorithm (Paturi, Pudlák, Saks, Zane)

- Add all resolvents of size at most $o(\log n)$.
- Execute PPZ.

We will not prove the improved upper bound.

Derandomization of PPZ

Permutations: Small sample space

We need a polynomial-size $\sigma \subset S_n$ s.t.

$$\forall \vec{s} \in [1..n]^k \quad \Pr_{\pi \in U(\sigma)} \{\pi(s_i) = \max_{1 \leqslant j \leqslant k} \pi(s_j)\} \geqslant 1/k.$$

It is enough to have $\pi(s_1), \ldots, \pi(s_j)$ mutually independent.

Such k-wise independent small sample spaces are known but are beyond the scope of this course.

Derandomization of PPZ

Derandomizing the guesses

Algorithm (PPZ, derandomized)

```
Try all assignments with ⩽ εn "1"s.

If it does not help, for all permutations in σ,
   For all strings s ∈ {0,1}^{n(1−ε/k)+1},
      Try to decode s (use it as random guesses).
```

- $H(\varepsilon) = -\varepsilon \log_2 \varepsilon - (1-\varepsilon)\log_2 \varepsilon$ is the binary entropy function, let $0 < \varepsilon \leqslant 1/2$.
- The first phase $\widetilde{O}(2^{H(\varepsilon)\cdot n})$ (use Stirling's formula).
- If it fails, there is an ε-isolated sat. assignment.
- The second phase takes $\widetilde{O}(2^{n(1-\varepsilon/k)})$.

- Take ε such that $H(\varepsilon) = 1 - \varepsilon/k$.
  As $k \to \infty$, it leads to an $\widetilde{O}(2^{n(1-c_k)})$ algorithms for $c_k - \frac{1}{2k} \to 0$.

Will not estimate this

What if it is not k-SAT?

Algorithm (Calabro, Impagliazzo, Paturi)

```
Clause-shortening (F,k,𝒜):                    // 𝒜 is a k-SAT algorithm
    if F is in k-CNF then return 𝒜(F)
    else (so there is C ∈ F s.t. |C| > k)
       take distinct ℓ_1,...,ℓ_k ∈ C
       if Clause-shortening((F ∪ {ℓ_1 ∨ ... ∨ ℓ_k}) \ {C}, k, 𝒜) =``yes''
          then return ``yes''
       elif Clause-shortening(F[ℓ̄_1,...,ℓ̄_k], k, 𝒜) = True
          then return ``yes''
       else return ``no''
```

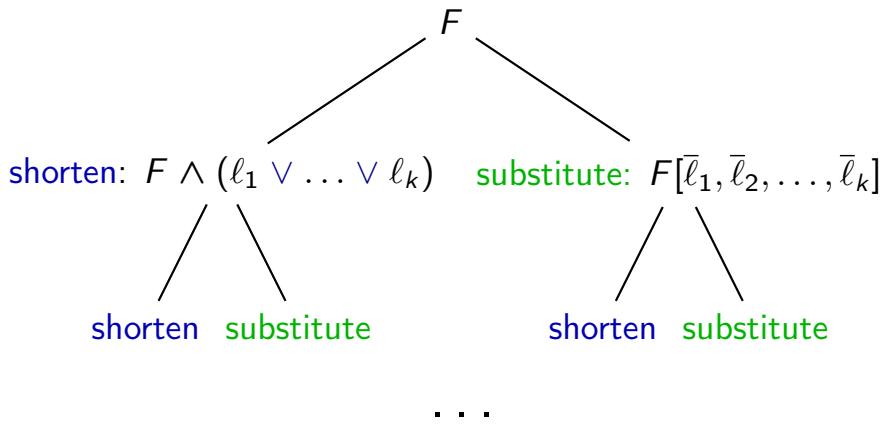
Input F: n variables, $m \geqslant n/k$ clauses.

Left branch: eliminate one long clause.
Right branch: eliminate k variables.

A path goes through L left branches and R right branches $\Rightarrow L \leqslant m$, $R \leqslant n/k$.

#paths with R r.b.: $\binom{L+R}{R} \leqslant \binom{m+n/k}{R}$,

$\mathcal{A}$ is applied to $n - kR$ vars k-CNF.



Leaves: applications of $\mathcal{A}$ to k-CNFs.

What if it is not k-SAT?

Algorithm (Calabro, Impagliazzo, Paturi)

```
Clause-shortening (F,k,𝒜):                    // 𝒜 is a k-SAT algorithm
    if F is in k-CNF then return 𝒜(F)
    else (so there is C ∈ F s.t. |C| > k)
       take distinct ℓ_1,...,ℓ_k ∈ C
       if Clause-shortening((F ∪ {ℓ_1 ∨ ... ∨ ℓ_k}) \ {C}, k, 𝒜) =``yes''
          then return ``yes''
       elif Clause-shortening(F[ℓ̄_1,...,ℓ̄_k], k, 𝒜) = True
          then return ``yes''
       else return ``no''
```

Input F: n variables, $m \geqslant n/k$ clauses.

Left branch: eliminate one long clause.
Right branch: eliminate k variables.

A path goes through L left branches and R right branches $\Rightarrow L \leqslant m$, $R \leqslant n/k$.

#paths with R r.b.: $\binom{L+R}{R} \leqslant \binom{m+n/k}{R}$,

$\mathcal{A}$ is applied to $n - kR$ vars k-CNF.

Time upper bound (ignore $\widetilde{O}(\cdot)$):

$2^{c_k n}$ for n vars

$$\sum_{R=0}^{n/k} \binom{m+n/k}{R} \cdot \text{time}_{\mathcal{A}}(m, n-kR)$$

$\sum \binom{N}{i} a^i = (1+a)^N$

$$\leqslant 2^{c_k n} \cdot \sum \binom{m+n/k}{R} \cdot 2^{-c_k k R}$$

$$\leqslant 2^{c_k n}(1 + 2^{-c_k k})^{m+n/k}$$

$$\leqslant 2^{c_k n} \exp(2^{-c_k k}(m + n/k))$$

$$\leqslant 2^{c_k \boldsymbol{n} + (2\boldsymbol{m} \log_2 \boldsymbol{e})/2^{c_k k}}$$

What if it is not k-SAT?

Algorithm (Calabro, Impagliazzo, Paturi)

```
Clause-shortening (F,k,𝒜):                    // 𝒜 is a k-SAT algorithm
    if F is in k-CNF then return 𝒜(F)
    else (so there is C ∈ F s.t. |C| > k)
       take distinct ℓ_1,...,ℓ_k ∈ C
       if Clause-shortening((F ∪ {ℓ_1 ∨ ... ∨ ℓ_k}) \ {C}, k, 𝒜) =``yes''
          then return ``yes''
       elif Clause-shortening(F[ℓ̄_1,...,ℓ̄_k], k, 𝒜) = True
          then return ``yes''
       else return ``no''
```

Input F: n variables, $m \geqslant n$ clauses, optimize k.

$c_k = 1 - 1/k$

k-SAT is solvable in time $\widetilde{O}\left(2^{n(1-1/k)}\right)$.

Choose $k = 2\log(m/n) + c$.

Our exponent $\leqslant c_k n + (4m)/2^{c_k k} = n(1-\frac{1}{k}) + \frac{4m}{2^{k-1}} = n(1-\frac{1}{2\log(m/n)+c} + \frac{1}{(m/n)\,2^{c-3}})$.

Choose c so that $2\log(m/n)+c < (m/n)2^{c-3}$.

In total: time $\widetilde{O}(2^{n\left(1-\frac{1}{O(\log(m/n))}\right)})$.

Time upper bound (ignore $\widetilde{O}(\cdot)$):

$2^{c_k n}$ for n vars

$$\sum_{R=0}^{n/k} \binom{m+n/k}{R} \cdot \text{time}_{\mathcal{A}}(m, n-kR)$$

$\sum \binom{N}{i} a^i = (1+a)^N$

$$\leqslant 2^{c_k n} \cdot \sum \binom{m+n/k}{R} \cdot 2^{-c_k k R}$$

$$\leqslant 2^{c_k n}(1 + 2^{-c_k k})^{m+n/k}$$

$$\leqslant 2^{c_k n} \exp(2^{-c_k k}(m + n/k))$$

$$\leqslant 2^{c_k \boldsymbol{n} + (2\boldsymbol{m}\log_2 \boldsymbol{e})/2^{c_k k}}$$

Randomized Local Search: another popular approach

Algorithm (GSAT, greedy algorithm by Bart Selman et al)

Input:

F in CNF, integers `MAXFLIPS` *and* `MAXTRIES`*.*

Output:

A truth assignment for the variables of F, or "no".

```
Repeat MAXTRIES times:
  — Pick an assignment A at random.
  — Repeat MAXFLIPS times:
      — If A satisfies F, then output A.
      — Choose a variable x in F such that A^x satisfies the
        maximal possible number of clauses of F.
      — A := A^x.
Output ''no''.
```

GSAT gets stuck: A lower bound $\Omega(2^n)$

$$\{x_1, \overline{x_2}\}$$
$$\{x_2, \overline{x_3}\}$$
$$\dots$$
$$\{x_n, \overline{x_1}\}$$

$$\{x_1, x_2\}$$

$$\{x_1, \quad \overline{x_2}, \dots, \overline{x_{n-3}}, \quad \overline{x_{n-2}}, \overline{x_{n-1}}\}$$
$$\{x_1, \quad \overline{x_2}, \dots, \overline{x_{n-3}}, \quad \overline{x_{n-1}}, \overline{x_n}\}$$
$$\{x_1, \quad \overline{x_2}, \dots, \overline{x_{n-3}}, \quad \overline{x_n}, \overline{x_{n-2}}\}$$

$$\{x_2, \quad \overline{x_3}, \dots, \overline{x_{n-2}}, \quad \overline{x_{n-1}}, \overline{x_n}\}$$
$$\{x_2, \quad \overline{x_3}, \dots, \overline{x_{n-2}}, \quad \overline{x_n}, \overline{x_1}\}$$
$$\{x_2, \quad \overline{x_3}, \dots, \overline{x_{n-2}}, \quad \overline{x_1}, \overline{x_{n-1}}\}$$
$$\dots$$
$$\{x_n, \quad \overline{x_1}, \dots, \overline{x_{n-4}}, \quad \overline{x_{n-3}}, \overline{x_{n-2}}\}$$
$$\{x_n, \quad \overline{x_1}, \dots, \overline{x_{n-4}}, \quad \overline{x_{n-2}}, \overline{x_{n-1}}\}$$
$$\{x_n, \quad \overline{x_1}, \dots, \overline{x_{n-4}}, \quad \overline{x_{n-1}}, \overline{x_{n-3}}\}$$

Satisfying assignment: all-1.

A at Hamming distance 1: falsify 4 clauses.
B at Hamming distance 2: falsify 2 or 3 clauses.
There x s.t. B^x, at distance 3, falsifies the same number of clauses (adjacent to exactly one of the wrong values and not $\in \{x_1, x_2\}$).

WalkSAT: a heuristic fix to the bug

Algorithm (WalkSAT, mixes greedy and random moves)

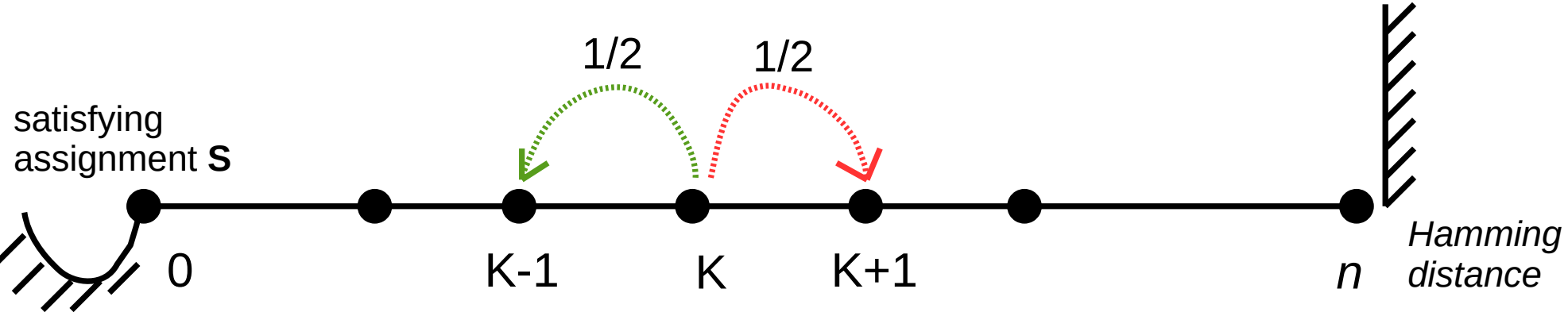
Input: *F in CNF, integers* `MAXFLIPS` *and* `MAXTRIES`*, probability p.*
Output: *A truth assignment for the variables of F, or "no".*

$\beta(A, x)$: the number of clauses satisfied by A but not by A^x

```
Repeat MAXTRIES times:
  — Pick an assignment A at random.
  — Repeat MAXFLIPS times:
      — If A satisfies F, then output A.
      — Choose an unsatisfied C ∈ F at random.
      — If there is x in C s.t.  β(A,x) = 0, then choose it
      — Else
      — -- with prob. p choose x from C to minimize β(A,x),
      — -- with prob. 1 − p choose a random x from C.
      — A := A^x.
Output ''no''.
```

Reminder: random walk for **2-SAT**

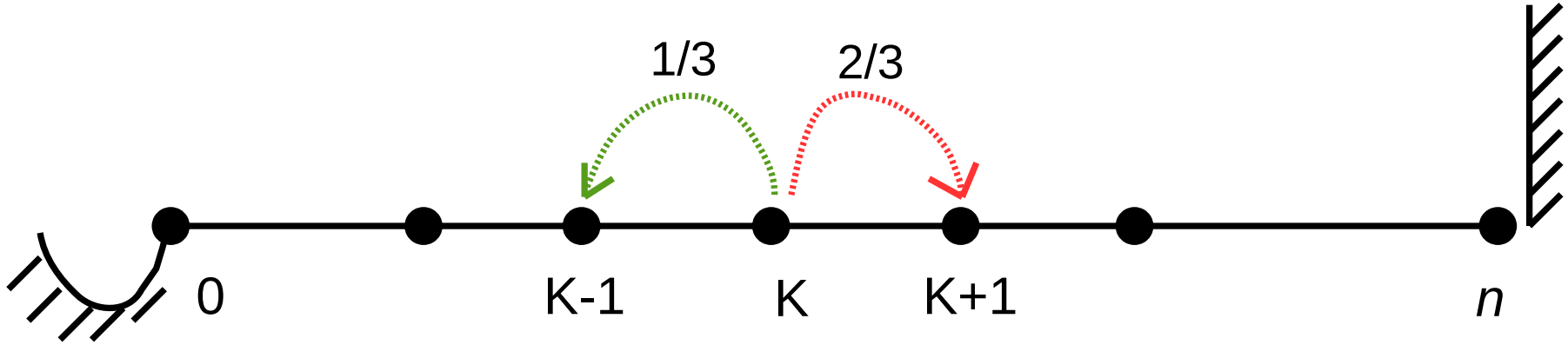
1. Pick a random assignment **A**.
2. While there is a clause *C* in *F* such that *C*[**A**] = false
   - Pick random variable *x* in *C*
   - **A** := **A**^{x} *// flip the value of x in **A***



The number of different bits in **A** vs **S** ---------->

Random walk for **3-SAT**

1. Pick a random assignment **A**.
2. While there is a clause *C* in *F* s.t. *C*[**A**] = false <u>(but at most *n* times)</u>
   - Pick random variable *x* in *C*
   - **A** := $\mathbf{A}^x$

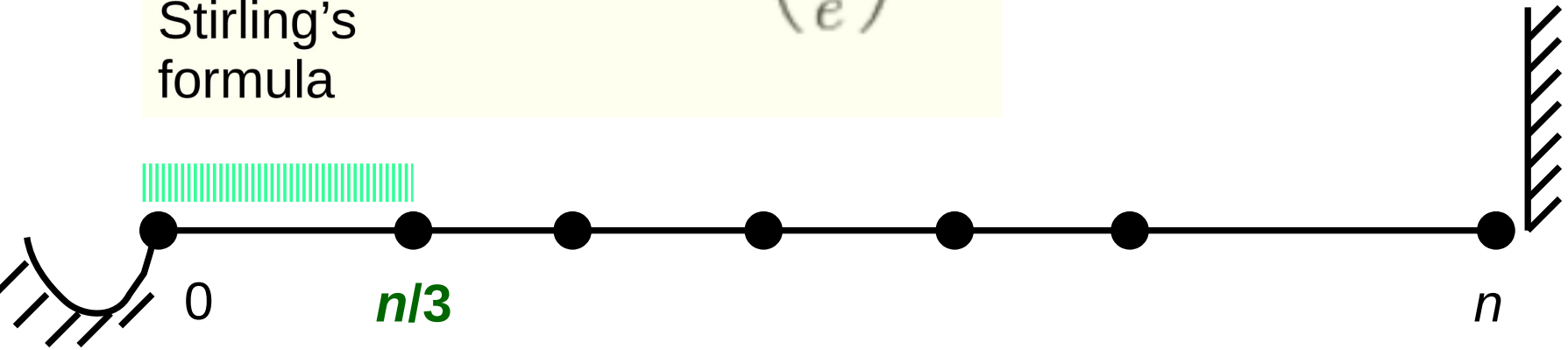


Compounds of the success: estimate two probabilities:
1. Choose a good starting assignment in the **state at most *n*/3**.
2. Reach 0 in **at most *n* steps.**

Part I: Land nearby

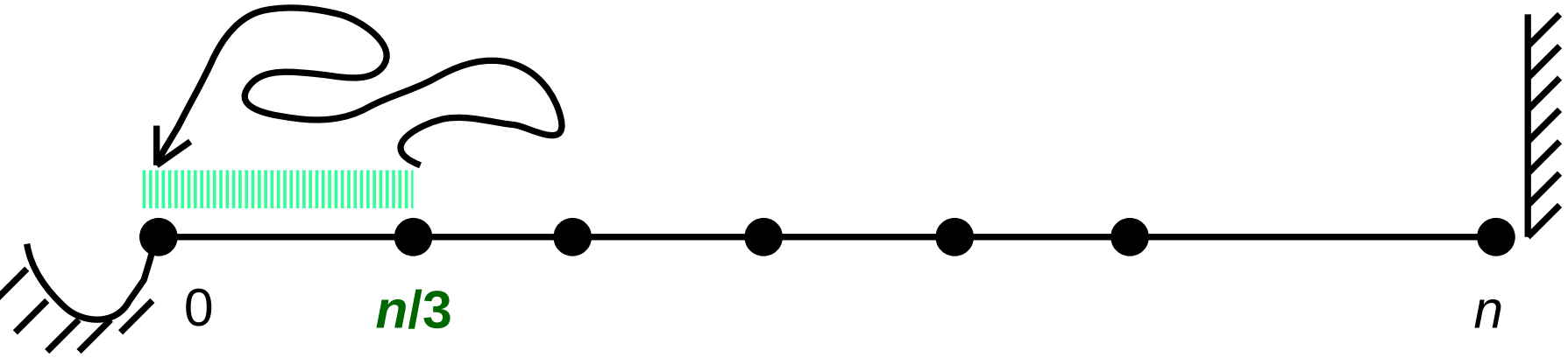
$$\binom{n}{n/3}/2^n \approx \left(\frac{3}{2^{2/3}}\right)^n/2^n = \left(\frac{3}{2^{5/3}}\right)^n$$

Recall Stirling's formula $n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$



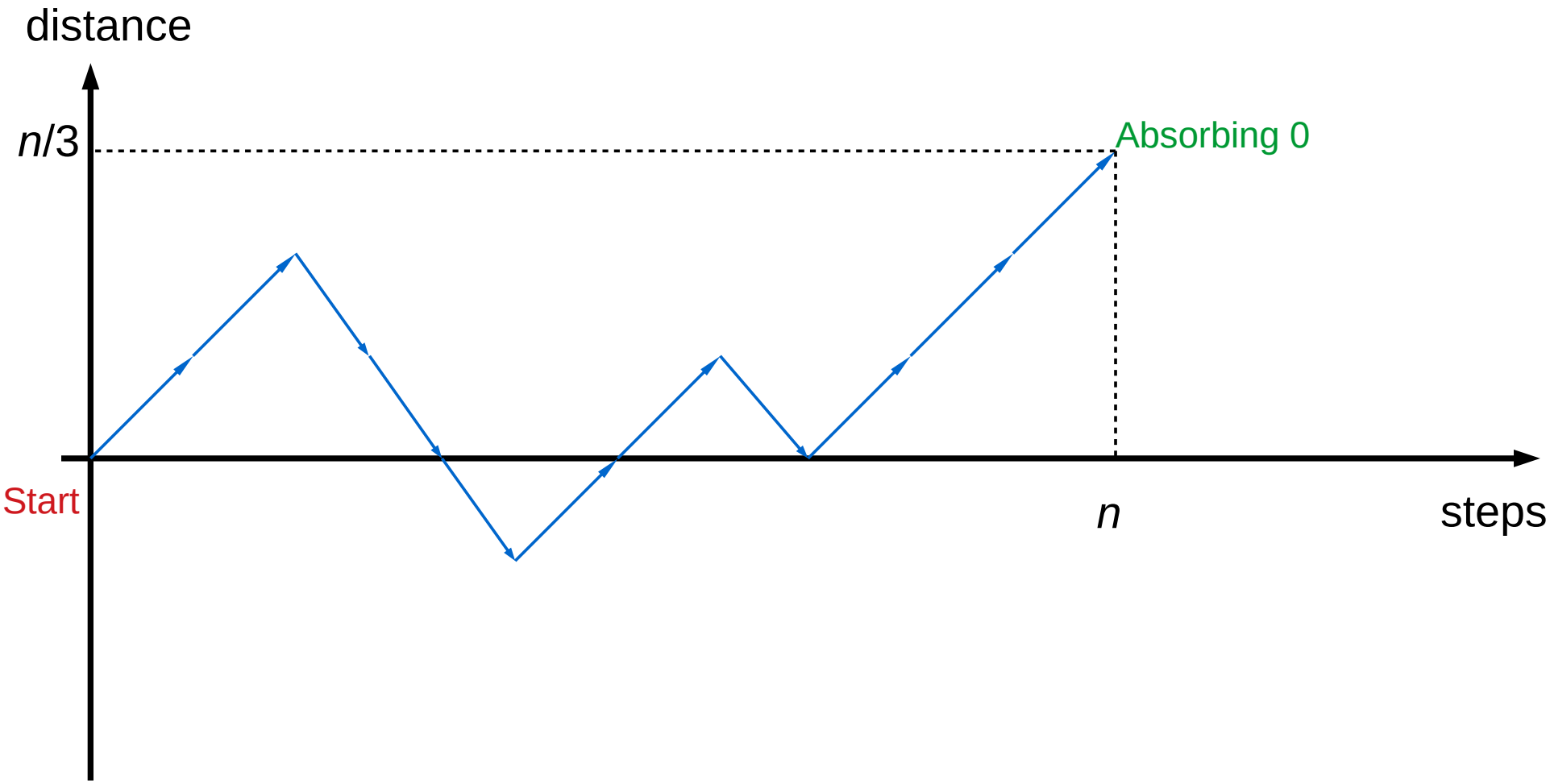
Part II: Reach 0 fast

How many paths do we have?



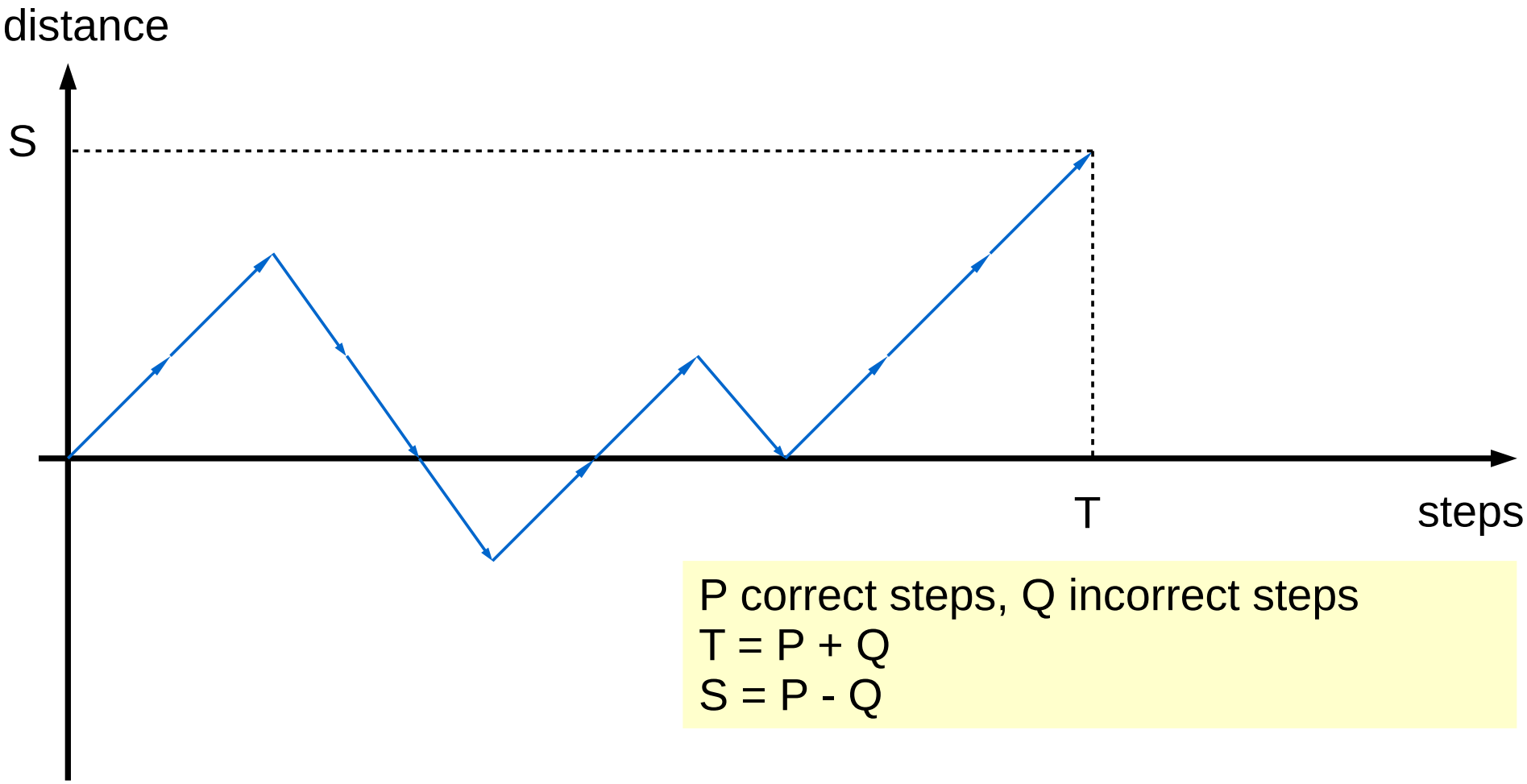
Part II: Reach 0 fast

#paths that hit the absorbing state 0



Part II: Reach 0 fast

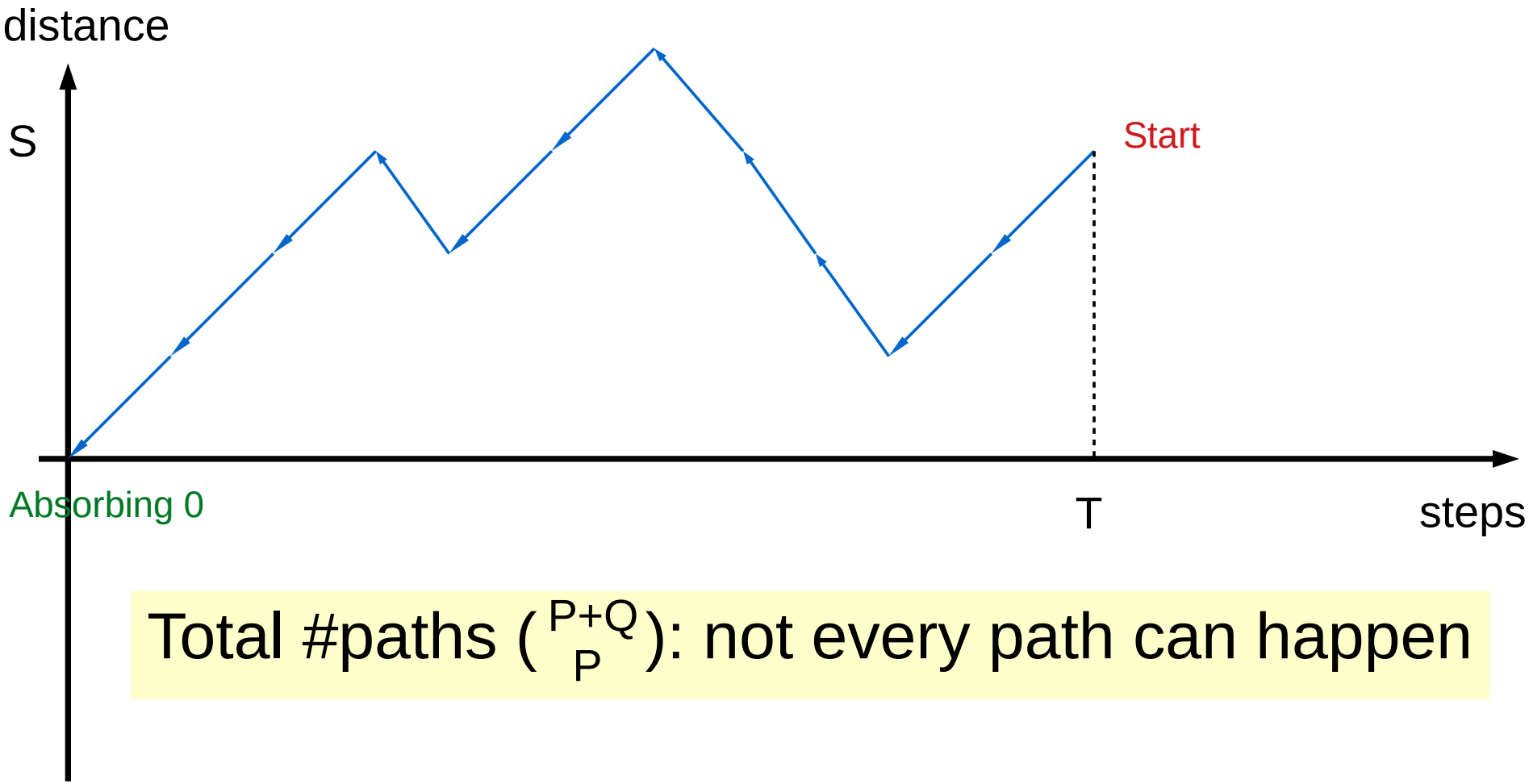
#paths that hit the absorbing state 0



P correct steps, Q incorrect steps
T = P + Q
S = P - Q

Part II: Reach 0 fast

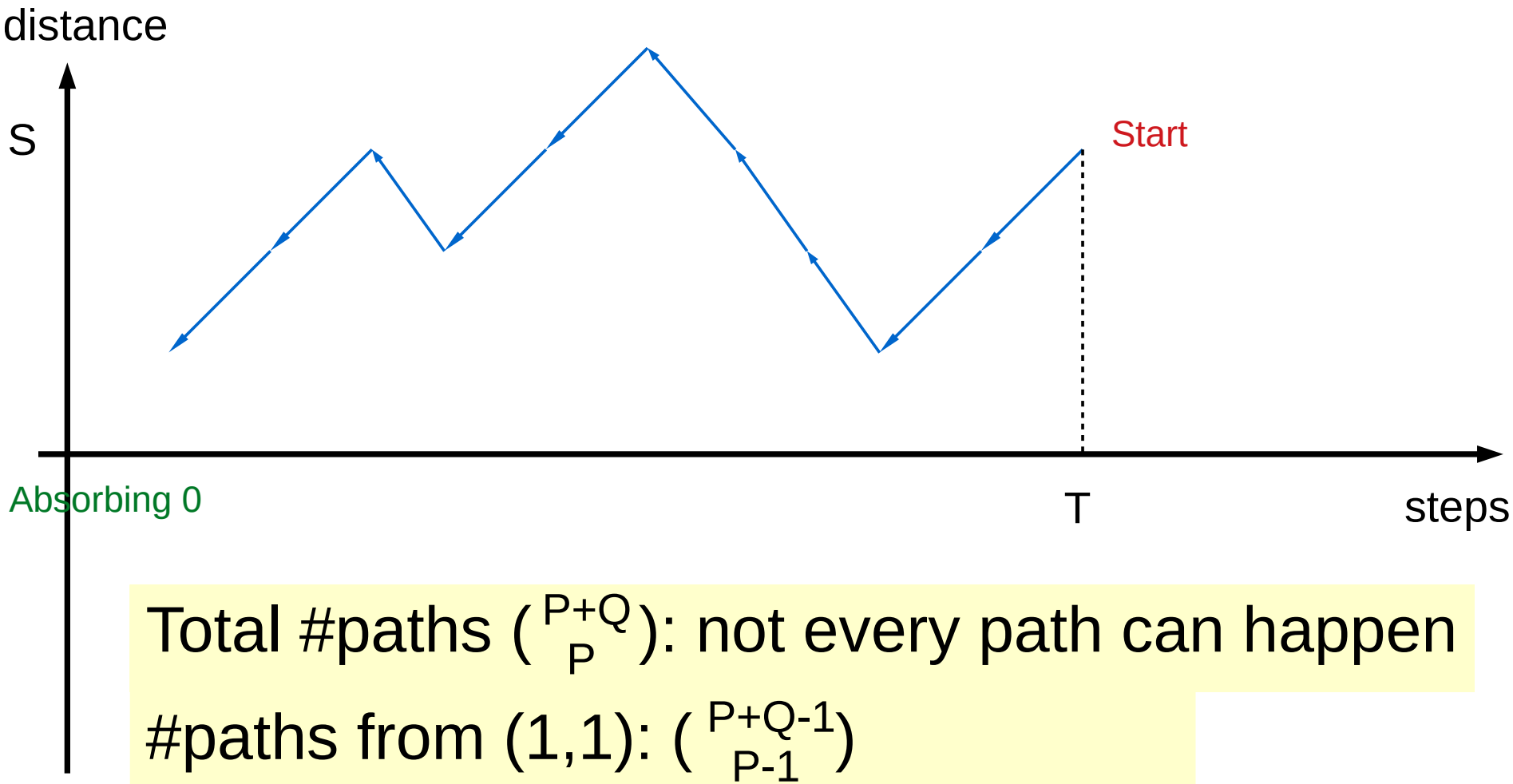
(we reversed the trip)



Total #paths $\binom{P+Q}{P}$: not every path can happen

Part II: Reach 0 fast

(we reversed the trip)

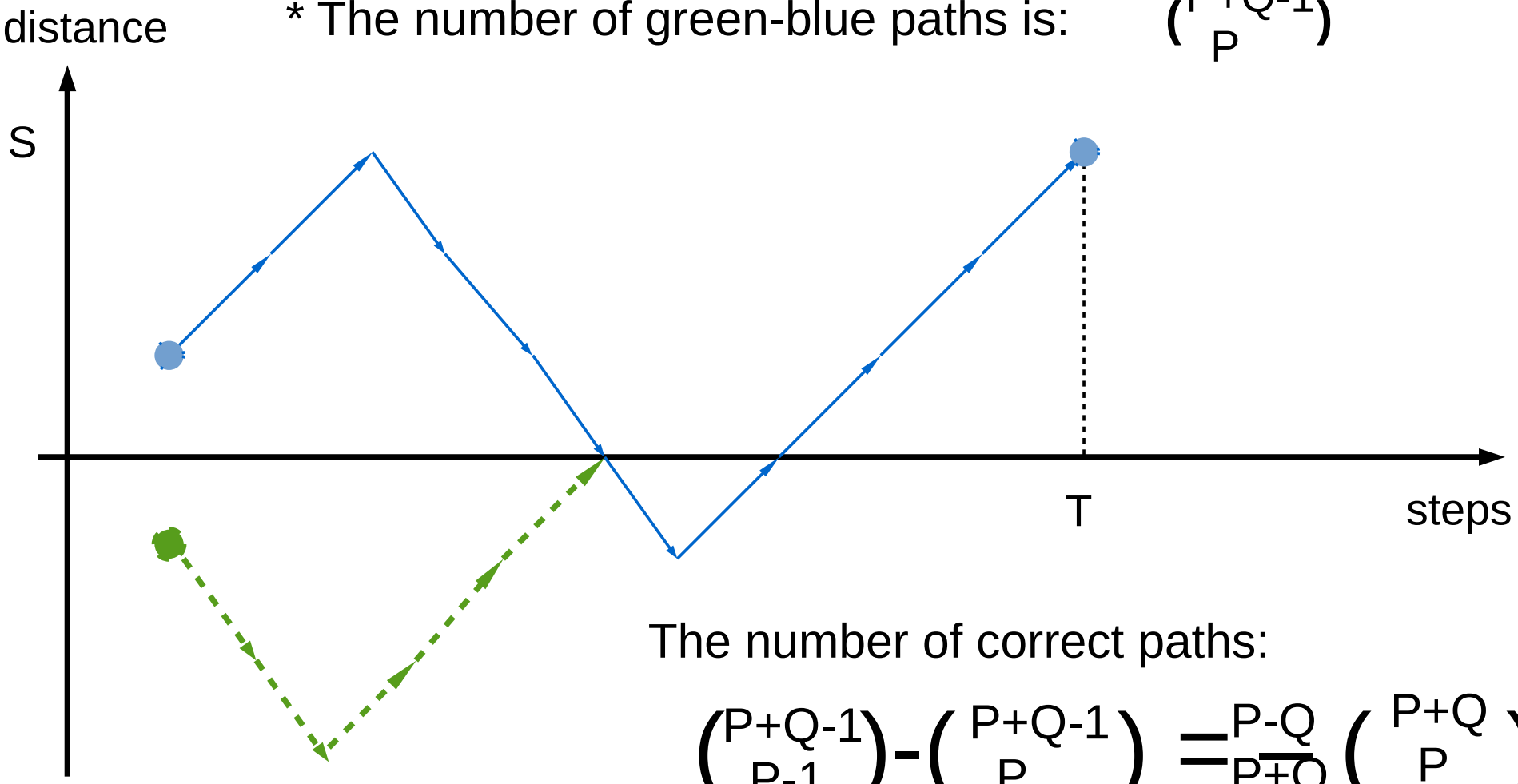


Total #paths $\binom{P+Q}{P}$: not every path can happen

#paths from (1,1): $\binom{P+Q-1}{P-1}$

Part II: Reach 0 fast

* A wrong path crosses the horizontal axe.
* Choose the leftmost point and reflect it.
* The number of green-blue paths is: $\binom{P+Q-1}{P}$



The number of correct paths:

$$\binom{P+Q-1}{P-1} - \binom{P+Q-1}{P} = \frac{P-Q}{P+Q}\binom{P+Q}{P}$$

Part II: Reach 0 fast

Thus the probability for Part II is...

$$\approx \binom{P+Q}{P}(1/3)^P(2/3)^Q =$$

$$\binom{n}{n/3}(1/3)^{2n/3}(2/3)^{n/3} \approx \frac{1}{2^{n/3}}$$

The overall probability of success:

$$\left(\frac{3}{2^{5/3}}\right)^n \bullet \frac{1}{2^{n/3}} = \left(\frac{3}{4}\right)^n$$

Running time: **$(4/3)^n$**

Random Walk: Generalizations

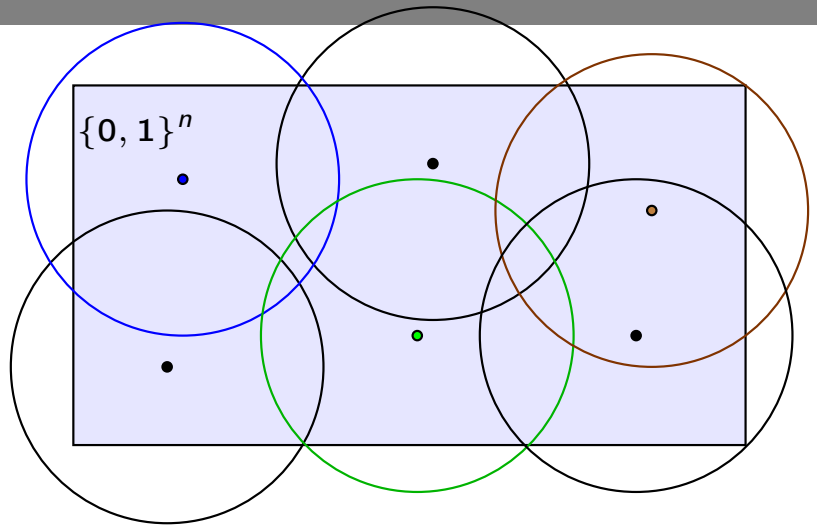
Direct generalizations:

- ***k*-SAT**, time $\widetilde{O}((2-\frac{2}{k})^n)$.
- **CSP**, time $\widetilde{O}((|D|(1-\frac{1}{d})^n)$ for domain D and arity d.

The author of all this: Uwe Schöning.

Derandomization: Deterministic local search

To start in a good position, try centers of Hamming balls of a certain radius R covering our search space.

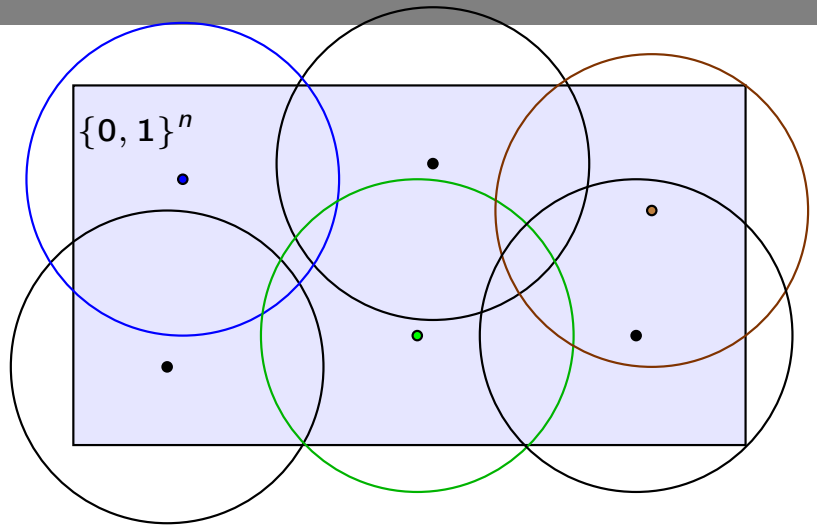


delayed

Let $0 < \rho < 1/2$. For every n, we can efficiently construct a covering code of length n, radius at most ρn, and cardinality $\widetilde{O}(2^{(1-H(\rho))n})$.

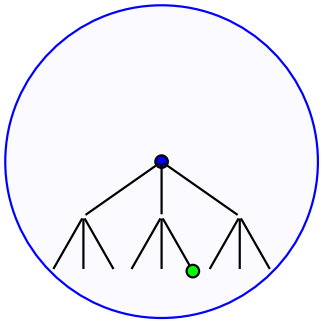
Derandomization: Deterministic local search

To start in a good position, try centers of Hamming balls of a certain radius R covering our search space.



delayed

Let $0 < \rho < 1/2$. For every n, we can efficiently construct a covering code of length n, radius at most ρn, and cardinality $\widetilde{O}(2^{(1-H(\rho))n})$.

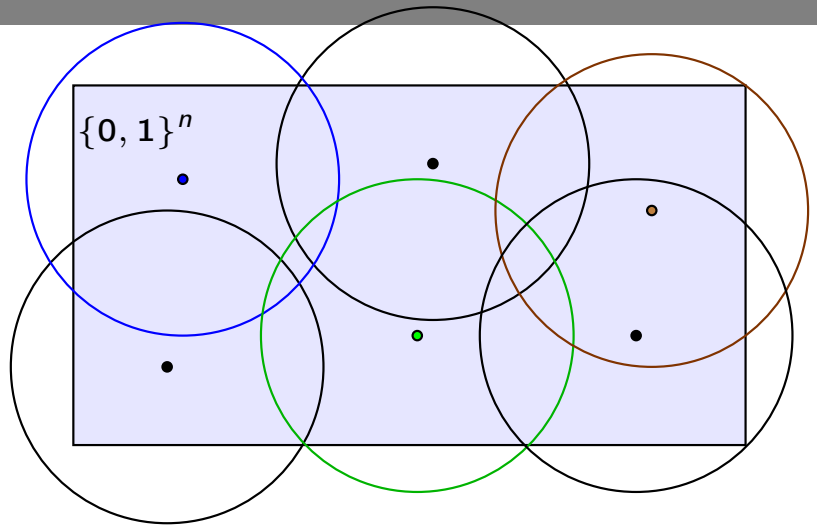


Start the search in a center of the ball.

When performing local search within a ball, make substitutions (we never come back) and stop at depth R, thus $\leqslant k^R$ leaves for k-CNF.

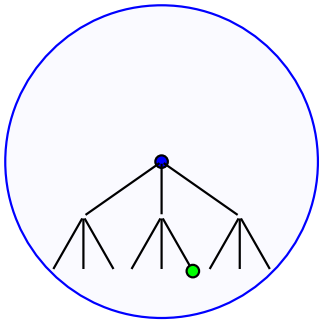
Derandomization: Deterministic local search

To start in a good position, try centers of Hamming balls of a certain radius R covering our search space.



delayed

Let $0 < \rho < 1/2$. For every n, we can efficiently construct a covering code of length n, radius at most ρn, and cardinality $\widetilde{O}(2^{(1-H(\rho))n})$.



Start the search in a center of the ball.

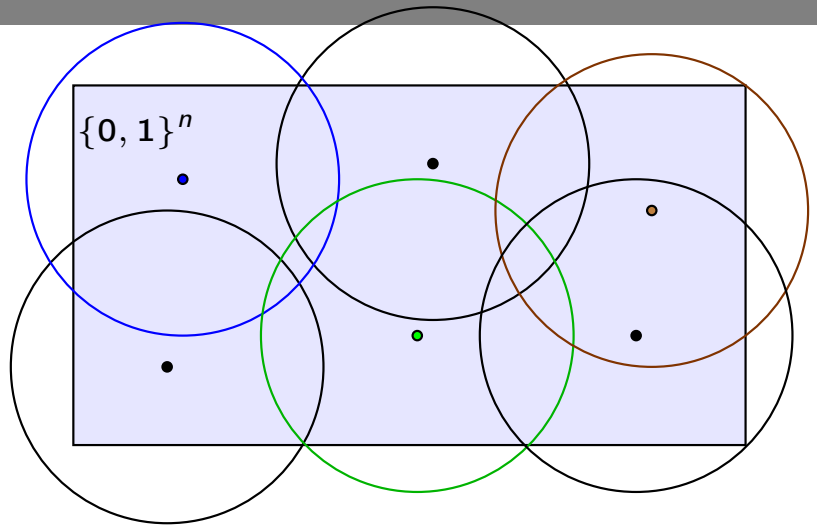
When performing local search within a ball, make substitutions (we never come back) and stop at depth R, thus $\leqslant k^R$ leaves for k-CNF.

Take it all together. Let $R = n/(k+1)$. Then the running time is $\widetilde{O}$ of

$$2^{(1-H(1/(k+1)))n} \cdot k^{n/(k+1)}$$
$$= 2^{(1-\log(k+1)/(k+1)+(1-1/(k+1))\log(1-1/(k+1)))n} \cdot 2^{n\log k/(k+1)}$$
$$= 2^{(1-\log(k+1)/(k+1)+(k/(k+1))\log(k/(k+1)))n} \cdot 2^{n\log k/(k+1)}$$
$$= 2^{n(1-\log(k+1)/(k+1)+\underline{(k/(k+1))\log k}-(k/(k+1))\log(k+1))+\underline{\log k/(k+1)})}$$
$$= 2^{n(1+\underline{\log k}-\log(k+1))} = (2-\tfrac{2}{k+1})^n .$$

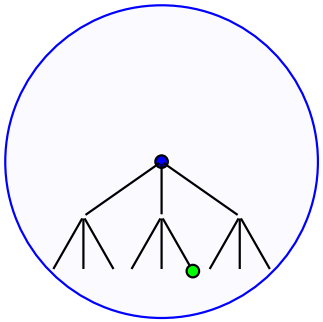
Derandomization: Deterministic local search

To start in a good position, try centers of Hamming balls of a certain radius R covering our search space.



delayed

Let $0 < \rho < 1/2$. For every n, we can efficiently construct a covering code of length n, radius at most ρn, and cardinality $\widetilde{O}(2^{(1-H(\rho))n})$.



Start the search in a center of the ball.

When performing local search within a ball, make substitutions (we never come back) and stop at depth R, thus $\leqslant k^R$ leaves for k-CNF.

Take it all together. Let $R = n/(k+1)$. Then the running time is $\widetilde{O}$ of

$$2^{(1-H(1/(k+1)))n} \cdot k^{n/(k+1)}$$
$$= 2^{(1-\log(k+1)/(k+1)+(1-1/(k+1))\log(1-1/(k+1)))n} \cdot 2^{n\log k/(k+1)}$$
$$= 2^{(1-\log(k+1)/(k+1)+(k/(k+1))\log(k/(k+1)))n} \cdot 2^{n\log k/(k+1)}$$
$$= 2^{n(1-\log(k+1)/(k+1)+\underline{(k/(k+1))\log k}-(k/(k+1))\log(k+1))+\underline{\log k/(k+1)})}$$
$$= 2^{n(1+\underline{\log k}-\log(k+1))} = (2-\tfrac{2}{k+1})^n .$$

It is possible to do even better!

How to construct a covering code of size $2^{(1-H(r/n))n}$?

Problem: too difficult to verify properties of such a HUGE set.
Solution: construct sets for small length d and take direct product.

$$\begin{pmatrix}00000\\01010\\10101\end{pmatrix} \cdot \begin{pmatrix}00000\\01010\\10101\end{pmatrix} \cdot \cdots \cdot \begin{pmatrix}00000\\01010\\10101\end{pmatrix}$$

Length: $n/d \times d = n$.

Radius: $n/d \cdot \rho d = \rho n$.

Cardinality: $\left(2^{\alpha d}\right)^{n/d} = 2^{\alpha n}$.

The existence of the (small) code:

Hamming distance

Like $\widetilde{O}$, but both $\leqslant$ and $\geqslant$

Let $V(d,r) = |\{A : \delta(A,\vec{0}) \leqslant r\}| = \widetilde{\Theta}(2^{H(r/d)d})$ be the volume of a ball of radius r.
There exists a code of length d, radius at most r, and size $\leqslant \lceil d \cdot 2^d / V(d,r) \rceil$.

- Choose $d \cdot 2^d / V(d,r)$ random strings $\in \{0,1\}^d$.
- Let $a \in \{0,1\}^d$. $\Pr_{b \in U(\{0,1\}^d)}\{a \text{ is in the ball centered at } b\} \geqslant V(d,r)/2^d$.
- Pr that a misses all our balls is $\left(1 - V(d,r)/2^d\right)^{d \cdot 2^d / V(n,r)} \leqslant \boldsymbol{e}^{-d}$.

How to construct a covering code of size $2^{(1-H(r/n))n}$?

Problem: too difficult to verify properties of such a HUGE set.
Solution: construct sets for small length d and take direct product.

$$\begin{pmatrix}00000\\01010\\10101\end{pmatrix}\cdot\begin{pmatrix}00000\\01010\\10101\end{pmatrix}\cdot\cdots\cdot\begin{pmatrix}00000\\01010\\10101\end{pmatrix}$$

Length: $n/d \times d = n$.

Radius: $n/d \cdot \rho d = \rho n$.

Cardinality: $\left(2^{\alpha d}\right)^{n/d} = 2^{\alpha n}$.

The existence of the (small) code:

How to find the small code:

- $d = O(1)$, then rounding errors add some ε to the exponent.
- $d = n/10$, then to construct the code we can use an approximation algorithm for Set Cover.

Details: exercise.

Hamming distance

Like $\widetilde{O}$, but both $\leqslant$ and $\geqslant$

Let $V(d,r) = |\{A : \delta(A,\vec{0}) \leqslant r\}| = \widetilde{\Theta}(2^{H(r/d)d})$ be the volume of a ball of radius r.
There exists a code of length d, radius at most r, and size $\leqslant \lceil d \cdot 2^d / V(d,r) \rceil$.

- Choose $d \cdot 2^d / V(d,r)$ random strings $\in \{0,1\}^d$.
- Let $a \in \{0,1\}^d$. $\Pr_{b \in U(\{0,1\}^d)}\{a \text{ is in the ball centered at } b\} \geqslant V(d,r)/2^d$.
- Pr that a misses all our balls is $\left(1 - V(d,r)/2^d\right)^{d \cdot 2^d / V(n,r)} \leqslant \boldsymbol{e}^{-d}$.

A note on quantum algorithms

Grover's quantum algorithm:
For any black-box $f\colon S \to \{0,1\}$,
finds $x \in S$ s.t. $f(x) = 1$ using $O(\sqrt{|S|})$ quantum queries to f.

Immediately gives an $\widetilde{O}(2^{\sqrt{n}})$-time algorithm.

Convert one of our improved algorithms into such f with smaller S,
and get an even better algorithm.

Takeaway (summary)

- Approaches to improve the exponent:
  - DPLL: divide and conquer.
    Extends to CDCL and Resolution, see "soon".
  - Clause shortening.
  - PPZ: random permutations (or not so random).
  - Random walk / local search (and search within balls).
  - Quantum: nothing special, speedup as usual.
- The first part of Homework Assignment #1 *(to be continued)*.

Takeaway (summary)

- Approaches to improve the exponent:
  - DPLL: divide and conquer.
    Extends to CDCL and Resolution, see "soon".
  - Clause shortening.
  - PPZ: random permutations (or not so random).
  - Random walk / local search (and search within balls).
  - Quantum: nothing special, speedup as usual.
- The first part of Homework Assignment #1 *(to be continued)*.
- Coming next: *How far can we go? The Exponential-Time Hypothesis.*

This slide is intentially left empty.